

페이지별 상세 기능 및 로직 분석

1. 메인 페이지 (HomePage)

서비스의 정체성을 알리고 사용자의 여정을 시작하는 페이지입니다.

- UI 구성:
 - Hero 섹션: **linear-gradient**를 활용한 시각적 강조와 함께 서비스의 핵심 가치를 텍스트로 전달합니다.
 - CTA 버튼: '수강신청 시작'과 '로그인' 버튼을 배치하여 사용자의 다음 행동을 유도합니다.
- 핵심 로직:
 - 특별한 상태 관리 없이 정적 정보를 전달하며, **Link** 컴포넌트를 통해 클라이언트 사이드 라우팅을 수행합니다.
- 콘텐츠: 서비스가 제공하는 4가지 핵심 기능(학생/교수 기능, 추천 로직, 챗봇)을 리스트 형태로 명시합니다.

2. 로그인 페이지 (LoginPage)

사용자 인증을 담당하며 세션을 생성하는 페이지입니다.

- UI 구성: **narrow** 클래스가 적용된 폭이 좁은 폼(Form) 구조입니다. 이메일과 비밀번호 입력 필드로 구성됩니다.
- 핵심 로직:
 - 입력 제어: **email, password** 상태(**useState**)를 통해 실시간으로 사용자 입력을 관리합니다.
 - 인증 프로세스: **demoUsers** 배열에서 입력값과 일치하는 객체를 **find** 함수로 탐색합니다.
 - 세션 처리: 인증 성공 시 **setSession**을 호출하여 **localStorage**에 유저 정보를 저장하고 메인으로 이동(**useNavigate**)합니다.
- 보안 특징: 데모 환경이므로 비밀번호를 평문으로 비교하며, 하단에 테스트 계정 정보를 힌트로 제공합니다.

3. 수강신청 목록 페이지 (CoursesPage)

전체 강의 데이터를 그리드 형태로 시각화하고 상호작용하는 페이지입니다.

- UI 구성: **grid** 레이아웃을 통해 강의 카드를 나열합니다. 각 카드에는 과목명, 교수, 학점, 요약 설명이 포함됩니다.
- 핵심 로직:
 - 상태 동기화: **setRefresh**라는 더미 상태를 사용하여 장바구니에 담은 즉시 버튼 UI가 '담김'으로 변하도록 리렌더링을 강제합니다.
 - 유효성 검사: **getSession()**을 통해 로그인 여부를 먼저 확인하고, 로그인이 안 되어 있을 경우 **alert**을 띄웁니다.

- 중복 체크: `getCart().some()` 로직을 통해 이미 담긴 과목은 버튼을 `disabled` 처리하여 데이터 무결성을 유지합니다.

4. 장바구니 페이지 (**CartPage**)

사용자가 선택한 과목을 검토하고 관리하는 페이지입니다.

- UI 구성: 선택된 과목들을 리스트화하며, 각 항목 옆에 '삭제' 버튼을 배치합니다.
- 핵심 로직:
 - 접근 제어: `RequireAuth`로 감싸져 있어 비로그인 사용자는 접근이 차단됩니다.
 - 삭제 기능: `removeCourseFromCart` 함수를 호출하여 `localStorage`에서 데이터를 제거하고 `setTick`을 통해 화면을 즉시 갱신합니다.
 - 조건부 렌더링: 장바구니가 비어있을 경우 "담긴 과목이 없습니다"라는 안내 메시지를 출력합니다.

5. 강의 상세 및 교재 상세 페이지 (**DetailPage**)

데이터 간의 관계성(Relation)을 보여주는 심층 페이지입니다.

- 강의 상세 (**CourseDetailPage**):
 - `useParams`로 URL에서 `courseId`를 추출해 해당 강의 정보를 표시합니다.
 - 데이터 매핑: `books.filter`를 사용해 현재 강의 ID와 연결된 모든 교재를 찾아 하단에 나열합니다.
- 교재 상세 (**BookDetailPage**):
 - 교재의 가격을 `toLocaleString()`을 통해 통화 형식으로 출력합니다.
 - 추천 알고리즘: 현재 교재의 `category`와 동일한 카테고리를 가진 다른 책들을 필터링하여 '추천 도서' 섹션에 노출합니다.

6. 공통 레이아웃 및 챗봇 (**Layout, ChatWidget**)

모든 페이지에서 유지되는 전역 컴포넌트입니다.

- 상단 바 (**Topbar**): `cart-updated` 이벤트를 리스닝하여 장바구니 숫자를 실시간으로 업데이트합니다.
- 챗봇 (**ChatWidget**):
 - 상태 관리: 대화 내역을 `lines` 배열에 저장하여 채팅창에 렌더링합니다.
 - 응답 로직: `chatbotReply` 함수 내부에 정의된 정규표현식 규칙에 따라 즉각적인 봇 응답을 생성합니다.

PRD (제품 요구 사항 명세서) 상세 버전

1. 제품 식별 (**Product Identification**)

- 제품명: 스마트 캠퍼스 (Smart Campus MVP)
- 버전: v1.0.0 (Client-side Only)
- 상태: 설계 및 프로토타입 완료

2. 페이지별 상세 요구 사항 (UI/UX Requirements)

페이지 ID	요구 사항 상세	비고
PG-HOME	- 서비스의 4대 핵심 가치 노출 - 비로그인 유저도 접근 가능해야 함	
PG-AUTH	- 유효하지 않은 이메일/비밀번호 입력 시 에러 메시지 출력 - 로그인 성공 후 세션은 <code>localStorage</code>에 유지	보안 주의
PG-LIST	- 수강 과목의 '담기' 상태는 실시간으로 반영되어야 함 - 상세 페이지로의 하이퍼링크 연결	
PG-CART	- 로그인 유저 전용 페이지 - 삭제 시 즉각적인 카운트 반영 및 리스트 업데이트	권한 체크 필수
PG-DETAIL	- 강의-교재, 교재-추천도서 간의 데이터 관계 시각화 - 없는 ID 접근 시 "찾을 수 없음" 안내	

CM-CHAT

정규식 기반

- 대화창 열기/닫기 토글 기능

- 과목 코드 기반의 교재 검색 기능 지원

Sheets로 내보내기

3. 데이터 명세 (Data Schema)

- **Course:** courseId, courseName, professor, category, credits, summary
- **Book:** bookId, title, author, category, courseId, summary, price
- **User:** name, email, role, password

4. 사용자 여정 (User Journey)

1. 메인 접속 → 서비스 확인
2. 수강신청 페이지 → 강의 탐색 및 '담기' 클릭
3. 로그인 유도 → 데모 계정으로 로그인 성공
4. 장바구니 이동 → 담긴 과목 확인 및 필요 시 삭제
5. 상세 페이지 탐색 → 특정 강의 클릭 후 관련 교재 정보 확인
6. 챗봇 질의 → 궁금한 점을 챗봇에게 물어봄 (예: "CS101 교재 뭐야?")